



University of Calabria

Department of Computer Engineering, Modelling, Electronics
and Systems Engineering (DIMES)

MSc in Robotics and Automation Engineering

Design and Analysis of Reinforcement Learning Agents in a Pac-Man Environment

Course:

Intelligent Systems for Robotics

Instructor:

Prof. Francesco Pupo

Student:

Ludovica Perroni

Student ID:

269200

Academic Year 2025–2026

Sommario

Questo progetto presenta lo sviluppo di un ambiente semplificato ispirato al gioco Pac-Man per lo studio di algoritmi di Reinforcement Learning. In particolare, vengono implementati e confrontati due algoritmi tabulari: Q-Learning e SARSA. L'agente controlla Pac-Man all'interno di una griglia bidimensionale, con l'obiettivo di raccogliere cibo evitando la collisione con un fantasma.

L'ambiente include elementi di stocasticità, dovuti sia al movimento del fantasma sia alla generazione casuale del cibo, rendendo il problema adatto all'apprendimento tramite interazione con l'ambiente. Durante il training gli agenti apprendono una politica decisionale utilizzando una strategia epsilon-greedy e una funzione di reward progettata per incentivare la sopravvivenza e la raccolta del cibo.

Le prestazioni ottenute vengono confrontate con quelle di un agente casuale utilizzando diverse metriche, tra cui reward media, numero medio di cibi raccolti, numero medio di step sopravvissuti e collision rate. I risultati mostrano che entrambi gli algoritmi di Reinforcement Learning riescono ad apprendere strategie efficaci, superando nettamente il comportamento casuale. Inoltre, il confronto evidenzia differenze coerenti con la teoria tra l'approccio off-policy del Q-Learning e quello on-policy di SARSA.

Indice

1	Introduzione	1
2	Descrizione dell'Ambiente	3
2.1	Struttura della griglia	3
2.2	Rappresentazione dello stato	3
2.3	Azioni disponibili	4
2.4	Movimento del fantasma	4
2.5	Condizioni di terminazione	4
3	Funzione di Reward	5
3.1	Reward per ogni step	5
3.2	Movimenti non validi	5
3.3	Reward shaping basato sulla distanza	5
3.4	Raccolta del cibo	6
3.5	Collisione con il fantasma	6
3.6	Considerazioni sulla reward	6
4	Algoritmi di Reinforcement Learning	7
4.1	Q-Table	7
4.2	Politica ϵ -greedy	7
4.3	Q-Learning	8
4.4	SARSA	8
4.5	Differenze tra Q-Learning e SARSA	8
5	Implementazione del Progetto	10
5.1	Tecnologie utilizzate	10
5.2	Struttura del progetto	10
5.3	Implementazione degli algoritmi di Reinforcement Learning	11
5.4	Gestione della simulazione	11
6	Training degli Agenti	12
6.1	Parametri di training	12
6.2	Monitoraggio del training	12
6.3	Andamento del training di Q-Learning	13
6.4	Andamento del training di SARSA	14
6.5	Considerazioni sul training	14
6.6	Video dimostrativo della simulazione	15

7	Valutazione delle Prestazioni	16
7.1	Metriche utilizzate	16
7.2	Risultati numerici	16
7.3	Confronto della reward media	17
7.4	Confronto del numero di cibi raccolti	17
7.5	Confronto della sopravvivenza	18
7.6	Confronto del collision rate	19
7.7	Confronto tra Q-Learning e SARSA	20
8	Discussione dei Risultati	21
8.1	Comportamento dell'agente casuale	21
8.2	Apprendimento degli agenti RL	21
8.3	Interpretazione del Q-value	22
8.4	Differenze tra Q-Learning e SARSA	22
8.5	Limiti del progetto	22
9	Conclusioni	23

Capitolo 1

Introduzione

Il Reinforcement Learning (RL) rappresenta una delle principali tecniche di apprendimento automatico utilizzate per la risoluzione di problemi decisionali sequenziali. A differenza dell'apprendimento supervisionato, in cui il modello apprende a partire da esempi etichettati, nel Reinforcement Learning un agente impara attraverso l'interazione diretta con un ambiente. Durante questo processo, l'agente osserva uno stato, seleziona un'azione e riceve una ricompensa che valuta la qualità della decisione presa. L'obiettivo è apprendere una politica ottimale in grado di massimizzare la reward cumulativa nel lungo termine.

Negli ultimi anni il Reinforcement Learning ha trovato applicazione in numerosi ambiti, tra cui robotica, videogiochi, sistemi autonomi e controllo intelligente. In particolare, i videogiochi rappresentano un contesto ideale per lo studio di algoritmi RL, poiché richiedono pianificazione, adattamento dinamico e gestione di obiettivi multipli.

In questo progetto viene sviluppato un ambiente semplificato ispirato al gioco Pac-Man. L'agente controlla Pac-Man all'interno di una griglia bidimensionale e deve raccogliere cibo evitando la collisione con un fantasma. Il problema presenta elementi di stocasticità, dovuti sia alla generazione casuale del cibo sia al comportamento non deterministico del fantasma, rendendo necessario l'apprendimento di strategie adattive.

Lo scopo del progetto è confrontare il comportamento di due algoritmi classici di Reinforcement Learning tabulare: Q-Learning e SARSA. Entrambi gli algoritmi utilizzano una Q-table per stimare il valore delle azioni nei diversi stati dell'ambiente, ma differiscono nella modalità di aggiornamento dei valori Q. Il confronto permette di evidenziare le differenze tra un approccio off-policy, rappresentato dal Q-Learning, e un approccio on-policy, rappresentato da SARSA.

Le prestazioni degli agenti vengono valutate confrontandole con quelle di un agente casuale attraverso diverse metriche, tra cui reward media, numero di cibi raccolti, numero medio di step sopravvissuti e tasso di collisione con il fantasma. Inoltre, vengono analizzati i grafici di training per osservare l'evoluzione dell'apprendimento durante gli episodi.

L'obiettivo finale del progetto è verificare la capacità degli algoritmi di Reinforcement Learning di apprendere strategie efficaci in un ambiente dinamico e parzialmente stocastico, evidenziandone punti di forza e differenze comportamentali.

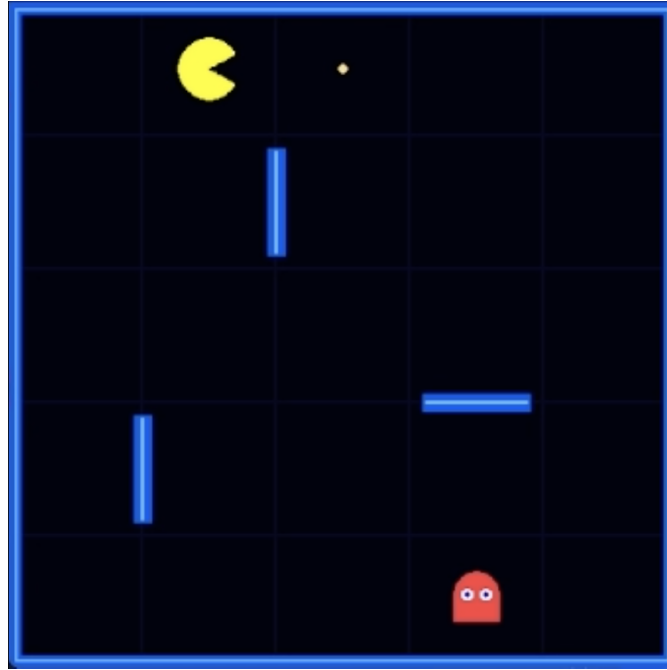


Figura 1.1: Ambiente Pac-Man utilizzato per l'addestramento degli agenti di Reinforcement Learning.

La Figura 1.1 mostra l'ambiente sviluppato per il progetto. L'agente Pac-Man deve raccogliere il cibo evitando il fantasma all'interno della griglia contenente ostacoli.

Capitolo 2

Descrizione dell'Ambiente

L'ambiente sviluppato per questo progetto rappresenta una versione semplificata del gioco Pac-Man, progettata specificamente per lo studio di algoritmi di Reinforcement Learning. L'ambiente è implementato come una griglia bidimensionale di dimensione 5×5 , all'interno della quale interagiscono tre elementi principali: Pac-Man, il fantasma e il cibo.

2.1 Struttura della griglia

La griglia è composta da 25 celle. Pac-Man parte sempre dalla posizione in alto a sinistra, mentre il fantasma parte dalla posizione in basso a destra. Il cibo viene invece generato casualmente all'inizio di ogni episodio in una posizione valida diversa da quelle occupate dagli altri elementi.

All'interno della griglia sono inoltre presenti alcuni muri che impediscono il passaggio diretto tra determinate celle. La presenza dei muri aumenta la complessità del problema, poiché limita i movimenti disponibili e obbliga l'agente a pianificare percorsi alternativi.

2.2 Rappresentazione dello stato

Lo stato dell'ambiente è rappresentato attraverso una tupla composta dalle coordinate di Pac-Man, del cibo e del fantasma:

$$s = (p_r, p_c, f_r, f_c, g_r, g_c) \quad (2.1)$$

dove:

- (p_r, p_c) rappresenta la posizione di Pac-Man;
- (f_r, f_c) rappresenta la posizione del cibo;
- (g_r, g_c) rappresenta la posizione del fantasma.

Questa rappresentazione permette all'agente di osservare completamente l'ambiente in ogni istante.

2.3 Azioni disponibili

L'agente può eseguire quattro azioni:

- movimento verso l'alto;
- movimento verso il basso;
- movimento verso sinistra;
- movimento verso destra.

Un movimento viene considerato valido solo se:

- rimane all'interno della griglia;
- non attraversa un muro.

In caso di movimento non valido, Pac-Man rimane nella posizione corrente e riceve una penalità.

2.4 Movimento del fantasma

Il fantasma utilizza una strategia semi-stocastica. Con probabilità del 70% si muove cercando di avvicinarsi a Pac-Man, mentre con probabilità del 30% effettua un movimento casuale tra quelli validi.

Questa scelta rende l'ambiente non completamente deterministico, poiché la stessa azione eseguita da Pac-Man può produrre risultati differenti a seconda del comportamento del fantasma.

2.5 Condizioni di terminazione

Ogni episodio termina quando si verifica una delle seguenti condizioni:

- Pac-Man entra in collisione con il fantasma;
- viene raggiunto il numero massimo di step consentiti.

Nel progetto il numero massimo di step per episodio è stato fissato a 100.

Capitolo 3

Funzione di Reward

La progettazione della funzione di reward rappresenta uno degli aspetti più importanti nel Reinforcement Learning, poiché influenza direttamente il comportamento appreso dall'agente. In questo progetto è stata definita una funzione di reward con l'obiettivo di incentivare Pac-Man a raccogliere il cibo, sopravvivere il più a lungo possibile ed evitare il fantasma.

3.1 Reward per ogni step

A ogni azione eseguita viene applicata una penalità base pari a:

$$r = -1 \tag{3.1}$$

Questa penalità ha lo scopo di scoraggiare movimenti inutili e incentivare l'agente a raggiungere rapidamente il cibo.

3.2 Movimenti non validi

Se Pac-Man tenta di attraversare un muro oppure uscire dai limiti della griglia, il movimento viene considerato non valido e viene applicata una penalità aggiuntiva:

$$r = -5 \tag{3.2}$$

In questo caso Pac-Man rimane nella stessa posizione.

3.3 Reward shaping basato sulla distanza

Per accelerare l'apprendimento è stata introdotta una strategia di reward shaping basata sulla distanza di Manhattan tra Pac-Man e il cibo.

La distanza di Manhattan è definita come:

$$d = |p_r - f_r| + |p_c - f_c| \tag{3.3}$$

Se dopo una mossa Pac-Man si avvicina al cibo, viene assegnato un bonus:

$$r = r + 1 \tag{3.4}$$

Se invece si allontana dal cibo, viene applicata una penalità:

$$r = r - 1 \tag{3.5}$$

Questa tecnica fornisce feedback più frequenti all'agente e facilita l'apprendimento di comportamenti utili anche nelle prime fasi del training.

3.4 Raccolta del cibo

Quando Pac-Man raggiunge la posizione del cibo, riceve una reward positiva:

$$r = r + 10 \tag{3.6}$$

Successivamente il cibo viene rigenerato casualmente in una nuova posizione valida della griglia.

3.5 Collisione con il fantasma

Se Pac-Man entra in collisione con il fantasma, l'episodio termina immediatamente e viene assegnata una penalità elevata:

$$r = -50 \tag{3.7}$$

Questa scelta permette all'agente di apprendere rapidamente che la collisione rappresenta un evento fortemente negativo da evitare.

3.6 Considerazioni sulla reward

La funzione di reward progettata combina penalità e ricompense con l'obiettivo di guidare gradualmente l'agente verso comportamenti efficaci. In particolare:

- le penalità per step incentivano percorsi brevi;
- il reward shaping accelera l'apprendimento;
- la reward positiva per il cibo incentiva il raggiungimento dell'obiettivo;
- la forte penalità per collisione favorisce strategie di sopravvivenza.

L'utilizzo del reward shaping si è rivelato particolarmente utile per stabilizzare il training e rendere più evidente il miglioramento delle performance durante gli episodi.

Capitolo 4

Algoritmi di Reinforcement Learning

Per risolvere il problema proposto sono stati implementati due algoritmi classici di Reinforcement Learning tabulare: Q-Learning e SARSA. Entrambi utilizzano una Q-table per memorizzare il valore associato a ciascuna coppia stato-azione, ma differiscono nel modo in cui aggiornano tali valori durante il training.

4.1 Q-Table

La Q-table rappresenta una struttura dati che associa a ogni stato dell'ambiente un insieme di valori Q, uno per ciascuna azione possibile. Il valore Q di una coppia (s, a) rappresenta una stima della reward cumulativa futura ottenibile eseguendo l'azione a nello stato s .

Formalmente:

$$Q(s, a) = \text{reward futura attesa eseguendo } a \text{ nello stato } s \quad (4.1)$$

Durante il training la Q-table viene aggiornata progressivamente in base alle esperienze raccolte dall'agente.

4.2 Politica ϵ -greedy

Entrambi gli algoritmi utilizzano una politica ϵ -greedy per bilanciare esplorazione e sfruttamento.

Con probabilità ϵ l'agente seleziona un'azione casuale, esplorando nuove possibilità. Con probabilità $1 - \epsilon$, invece, seleziona l'azione con valore Q massimo nello stato corrente.

Formalmente:

$$a = \begin{cases} \text{azione casuale} & \text{con probabilità } \epsilon \\ \arg \max_a Q(s, a) & \text{con probabilità } 1 - \epsilon \end{cases} \quad (4.2)$$

Nel progetto è stato utilizzato un valore costante:

$$\epsilon = 0.1 \quad (4.3)$$

4.3 Q-Learning

Il Q-Learning è un algoritmo off-policy. Questo significa che l'aggiornamento della Q-table viene effettuato assumendo che, nello stato successivo, venga scelta l'azione ottimale, indipendentemente dalla politica realmente utilizzata durante l'esplorazione.

La regola di aggiornamento è:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (4.4)$$

dove:

- α rappresenta il learning rate;
- γ rappresenta il discount factor;
- r è la reward ricevuta;
- s' è lo stato successivo.

Il termine:

$$\max_{a'} Q(s', a') \quad (4.5)$$

rappresenta la migliore reward futura stimata nello stato successivo.

Grazie a questa caratteristica, il Q-Learning tende a sviluppare strategie più aggressive e orientate alla massimizzazione della reward.

4.4 SARSA

SARSA è invece un algoritmo on-policy. A differenza del Q-Learning, aggiorna la Q-table utilizzando il valore dell'azione effettivamente scelta dalla politica corrente nello stato successivo.

La regola di aggiornamento è:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)] \quad (4.6)$$

In questo caso, l'aggiornamento dipende direttamente dall'azione realmente eseguita nel passo successivo.

Poiché SARSA tiene conto anche delle azioni esplorative introdotte dalla politica ϵ -greedy, tende generalmente ad apprendere strategie più conservative rispetto al Q-Learning.

4.5 Differenze tra Q-Learning e SARSA

La principale differenza tra i due algoritmi riguarda il modo in cui stimano il valore futuro.

- Q-Learning utilizza il massimo valore Q possibile nello stato successivo, assumendo sempre una scelta ottimale.
- SARSA utilizza il valore dell'azione effettivamente scelta dalla politica corrente.

Di conseguenza:

- Q-Learning tende a essere più ottimista e aggressivo;
- SARSA tende a essere più prudente e stabile.

Nel contesto del progetto, questa differenza si riflette nelle performance finali degli agenti e nel loro comportamento durante il training.

Capitolo 5

Implementazione del Progetto

Il progetto è stato sviluppato interamente in Python, utilizzando diverse librerie per la gestione dell'ambiente, dell'addestramento degli agenti e della visualizzazione grafica.

5.1 Tecnologie utilizzate

Le principali librerie utilizzate sono:

- **NumPy**: utilizzata per la gestione della Q-table e delle operazioni numeriche;
- **Pygame**: utilizzata per la realizzazione grafica dell'ambiente Pac-Man e per la simulazione interattiva;
- **Matplotlib**: utilizzata per la generazione dei grafici di training e dei confronti tra gli agenti;
- **JSON e CSV**: utilizzati per il salvataggio dei risultati sperimentali.

5.2 Struttura del progetto

Il progetto è organizzato in più file Python, ciascuno dedicato a una specifica funzionalità. In particolare:

- `environment.py` definisce l'ambiente Pac-Man;
- `train_qlearning.py` implementa l'algoritmo Q-Learning;
- `train_sarsa.py` implementa l'algoritmo SARSA;
- `visualizer.py` gestisce la visualizzazione grafica della simulazione;
- `save_results.py` salva grafici e metriche finali;
- `main.py` esegue il training e la valutazione degli agenti.

5.3 Implementazione degli algoritmi di Reinforcement Learning

Gli algoritmi Q-Learning e SARSA sono stati implementati manualmente in Python, senza utilizzare librerie esterne di Reinforcement Learning o framework predefiniti.

In particolare, sono stati sviluppati direttamente:

- la rappresentazione della Q-table;
- la politica epsilon-greedy;
- le regole di aggiornamento dei valori Q;
- il ciclo di training episodico;
- la gestione dell'interazione tra agente e ambiente.

La Q-table è stata implementata tramite un dizionario Python, nel quale a ogni stato dell'ambiente è associato un vettore contenente i valori Q relativi alle azioni possibili.

L'aggiornamento dei valori Q avviene secondo le equazioni classiche degli algoritmi Q-Learning e SARSA descritte nei capitoli precedenti.

5.4 Gestione della simulazione

La simulazione viene eseguita episodicamente. A ogni episodio:

1. l'ambiente viene inizializzato;
2. Pac-Man interagisce con la griglia;
3. il fantasma si muove secondo una politica semi-stocastica;
4. l'agente aggiorna la Q-table;
5. vengono salvate le metriche di training.

Durante la fase finale del progetto è stata inoltre implementata una visualizzazione grafica in tempo reale, utile per osservare qualitativamente il comportamento degli agenti addestrati.

Capitolo 6

Training degli Agenti

Il processo di training consiste nell'interazione ripetuta tra l'agente e l'ambiente. Durante ogni episodio, Pac-Man osserva lo stato corrente, seleziona un'azione tramite la politica ϵ -greedy e aggiorna la Q-table in base alla reward ricevuta e allo stato successivo.

6.1 Parametri di training

Entrambi gli algoritmi sono stati addestrati utilizzando gli stessi parametri, in modo da rendere il confronto il più possibile equo.

I parametri utilizzati sono riportati nella Tabella 6.1.

Parametro	Valore
Numero di episodi	10000
Learning rate α	0.1
Discount factor γ	0.95
Epsilon ϵ	0.1
Step massimi per episodio	100

Tabella 6.1: Parametri utilizzati durante il training.

6.2 Monitoraggio del training

Durante il training sono state salvate diverse informazioni per monitorare l'apprendimento degli agenti:

- reward totale ottenuta in ogni episodio;
- media cumulativa delle reward;
- media mobile delle reward;
- andamento del valore Q associato allo stato iniziale.

La media cumulativa permette di osservare il miglioramento complessivo dell'agente nel corso degli episodi, mentre la media mobile consente di ridurre il rumore presente nella reward dei singoli episodi e rendere più leggibile il trend generale.

6.3 Andamento del training di Q-Learning

La Figura 6.1 mostra l'andamento del training dell'agente Q-Learning.

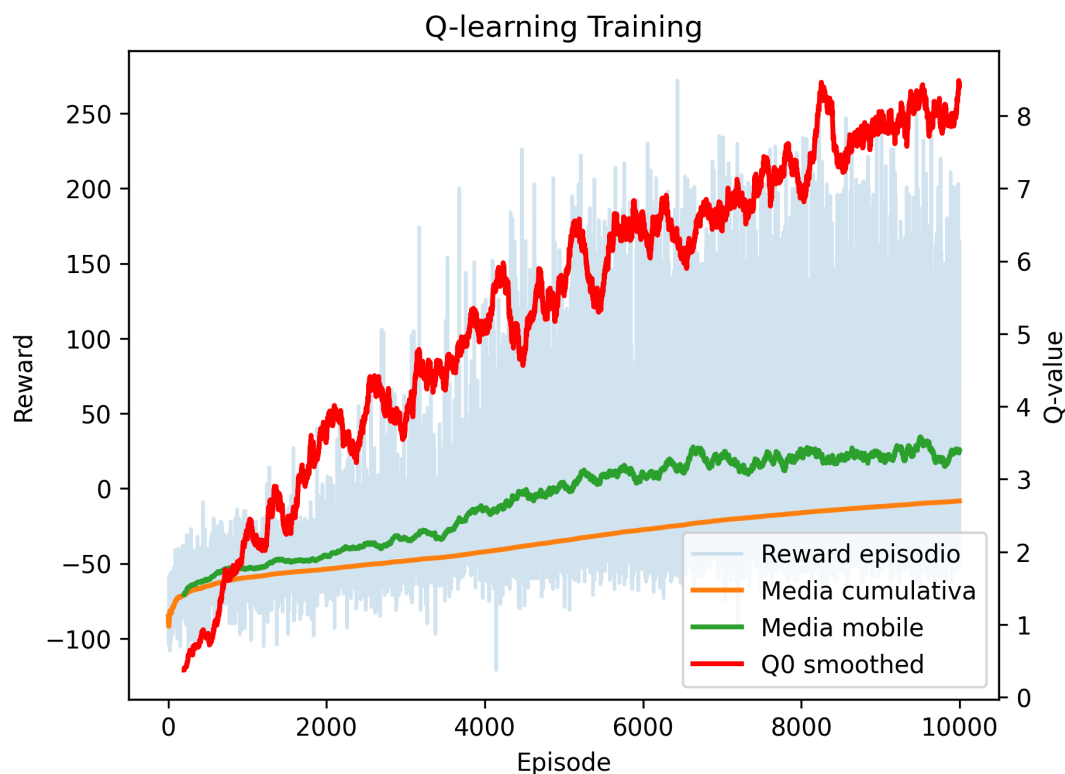


Figura 6.1: Andamento della reward e del Q-value durante il training dell'agente Q-Learning

Nel grafico è possibile osservare che:

- la reward dei singoli episodi presenta un andamento molto rumoroso;
- la media cumulativa cresce progressivamente;
- la media mobile mostra un miglioramento evidente delle performance;
- il valore Q dello stato iniziale tende ad aumentare durante il training.

L'elevata variabilità della reward è dovuta alla natura stocastica dell'ambiente. In particolare, la posizione del cibo cambia casualmente a ogni episodio e il fantasma utilizza una politica di movimento parzialmente casuale.

Il valore Q monitorato non converge perfettamente a un valore costante. Questo comportamento è plausibile perché lo stato iniziale dell'episodio non è sempre identico: la posizione del cibo varia casualmente e l'agente continua a esplorare tramite la politica ϵ -greedy. Pertanto, il Q-value deve essere interpretato come un indicatore generale dell'apprendimento della Q-table, piuttosto che come una misura esatta della convergenza.

6.4 Andamento del training di SARSA

La Figura 6.2 mostra il training dell'agente SARSA.

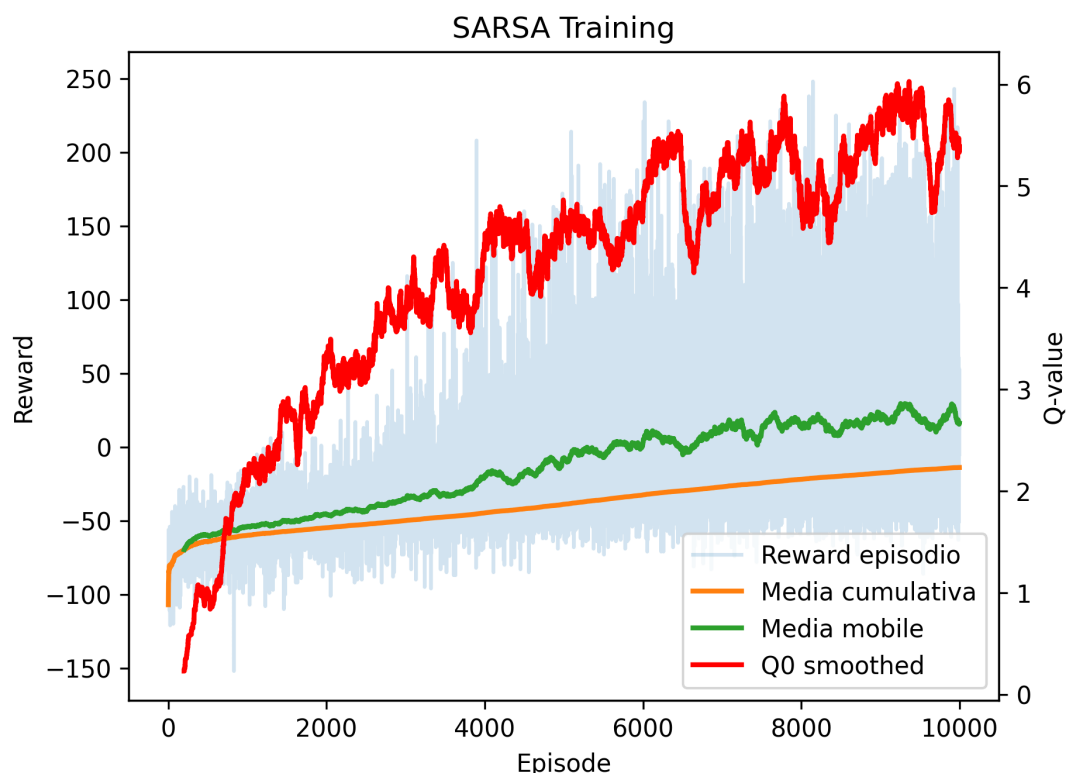


Figura 6.2: Andamento della reward e del Q-value durante il training dell'agente SARSA

Anche SARSA mostra un miglioramento progressivo delle performance durante il training. La media mobile cresce gradualmente e raggiunge valori elevati nelle fasi finali dell'addestramento.

Rispetto al Q-Learning, SARSA presenta un comportamento leggermente più conservativo. Questo risultato è coerente con la natura on-policy dell'algoritmo, che aggiorna la Q-table considerando le azioni realmente selezionate dalla politica corrente, comprese quelle esplorative.

6.5 Considerazioni sul training

I grafici mostrano che entrambi gli algoritmi riescono ad apprendere strategie efficaci per il problema proposto. L'aumento della reward media e la riduzione delle collisioni durante gli episodi indicano che gli agenti imparano progressivamente a:

- evitare il fantasma;
- raggiungere rapidamente il cibo;
- sopravvivere per un numero elevato di step.

Nonostante il rumore presente nei grafici, dovuto alla stocasticità dell'ambiente, il trend generale conferma la presenza di apprendimento.

6.6 Video dimostrativo della simulazione

Per osservare qualitativamente il comportamento degli agenti addestrati, è stato realizzato un video dimostrativo della simulazione dell'ambiente Pac-Man.

Nel video è possibile osservare:

- il comportamento dell'agente addestrato tramite Q-Learning;
- il comportamento dell'agente addestrato tramite SARSA;
- la capacità degli agenti di evitare il fantasma;
- la raccolta del cibo durante l'esecuzione.

Il video completo è disponibile tramite il seguente QR code:



[Repository GitHub del progetto](#)

Capitolo 7

Valutazione delle Prestazioni

Dopo il training, gli agenti Q-Learning e SARSA sono stati valutati eseguendo episodi di test senza apprendimento aggiuntivo. Durante la fase di valutazione gli agenti utilizzano una politica greedy, selezionando sempre l'azione con valore Q massimo nello stato corrente.

Le prestazioni ottenute sono state confrontate con quelle di un agente casuale, che seleziona le azioni in modo completamente randomico.

7.1 Metriche utilizzate

Per confrontare gli agenti sono state utilizzate le seguenti metriche:

- **Reward media:** misura la qualità complessiva del comportamento dell'agente;
- **Numero medio di cibi raccolti:** indica la capacità dell'agente di raggiungere l'obiettivo principale;
- **Numero medio di step:** misura la capacità di sopravvivenza;
- **Collision rate:** rappresenta la frequenza delle collisioni con il fantasma.

La valutazione è stata effettuata su 100 episodi per ciascun agente.

7.2 Risultati numerici

La Tabella 7.1 riassume i risultati ottenuti.

Agente	Reward media	Cibo medio	Step medi	Collision rate
Random	-72.56	0.15	11.91	1.00
Q-Learning	179.44	21.14	97.46	0.05
SARSA	182.43	21.49	97.35	0.04

Tabella 7.1: Confronto delle prestazioni degli agenti.

Nel confronto finale, entrambi gli algoritmi mostrano prestazioni molto simili e nettamente superiori rispetto all'agente casuale. In questo specifico training, SARSA ottiene una reward media leggermente superiore e un collision rate leggermente inferiore rispetto a Q-Learning.

Questo comportamento è plausibile poiché SARSA, essendo un algoritmo on-policy, tende a sviluppare strategie più conservative e stabili, particolarmente vantaggiose in ambienti stocastici come quello utilizzato nel progetto. Q-Learning, invece, mantiene un comportamento leggermente più aggressivo e orientato alla massimizzazione della reward futura.

7.3 Confronto della reward media

La Figura 7.1 mostra il confronto della reward media tra i diversi agenti.

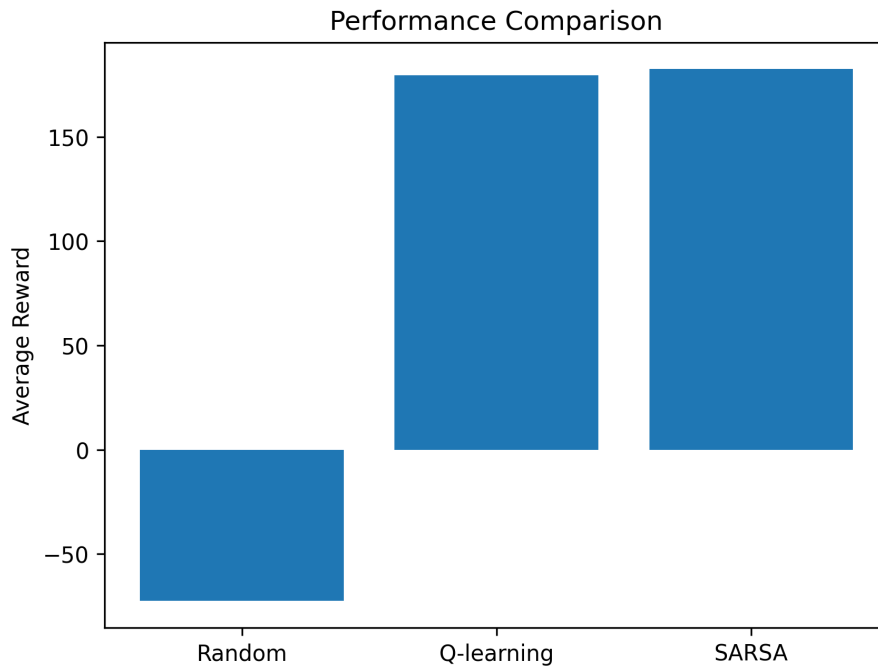


Figura 7.1: Confronto della reward media ottenuta dagli agenti durante la fase di valutazione.

L'agente casuale ottiene una reward media fortemente negativa, poiché tende a collidere rapidamente con il fantasma senza raccogliere quantità significative di cibo.

Gli agenti Q-Learning e SARSA ottengono invece reward medie elevate, dimostrando di aver appreso strategie efficaci.

7.4 Confronto del numero di cibi raccolti

La Figura 7.2 confronta il numero medio di cibi raccolti.

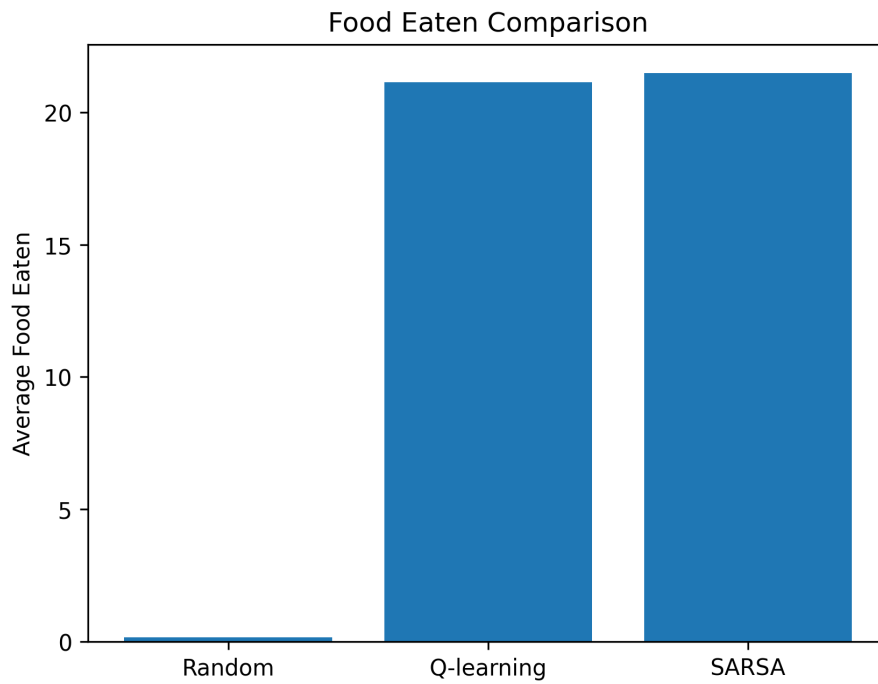


Figura 7.2: Confronto del numero medio di cibi raccolti dagli agenti durante gli episodi di test.

Gli agenti addestrati riescono a raccogliere una quantità molto maggiore di cibo rispetto all'agente random. Questo comportamento indica che gli algoritmi hanno appreso politiche in grado di raggiungere rapidamente gli obiettivi presenti nell'ambiente.

7.5 Confronto della sopravvivenza

La Figura 7.3 mostra il numero medio di step sopravvissuti.

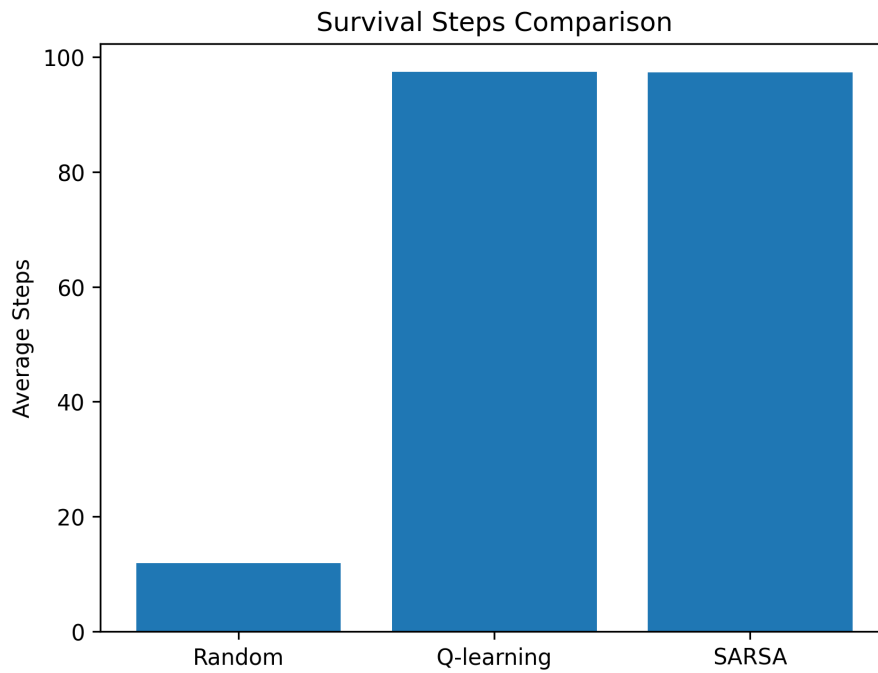


Figura 7.3: Confronto del numero medio di step sopravvissuti dagli agenti.

L'agente casuale sopravvive mediamente per pochi step, mentre gli agenti RL riescono quasi sempre a raggiungere il limite massimo di 100 step per episodio.

Questo risultato suggerisce che gli agenti hanno imparato strategie efficaci per evitare il fantasma.

7.6 Confronto del collision rate

La Figura 7.4 mostra il tasso di collisione con il fantasma.

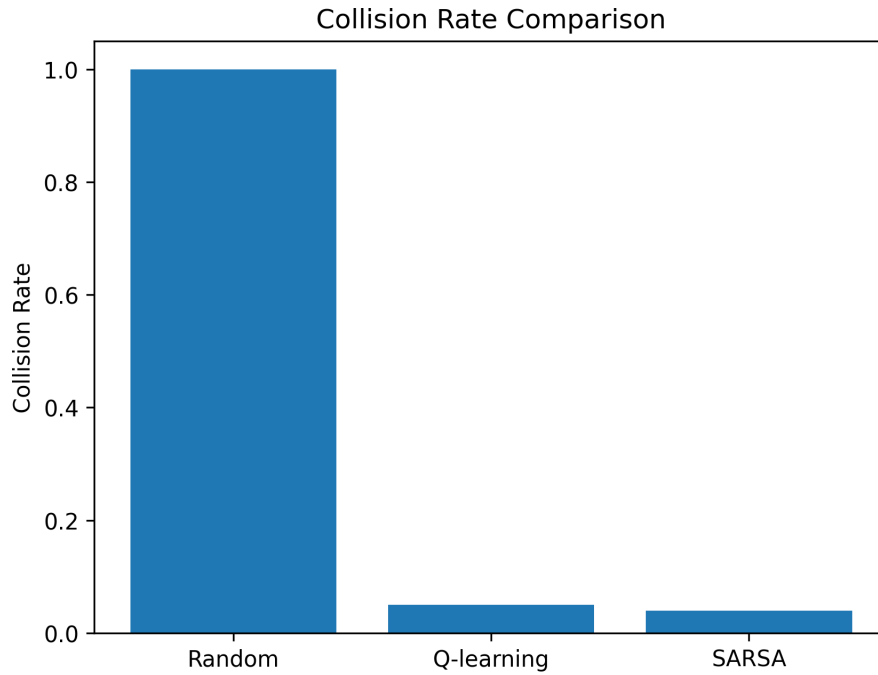


Figura 7.4: Confronto della frequenza di collisione con il fantasma tra gli agenti.

L'agente casuale presenta un collision rate pari al 100%, indicando che termina praticamente tutti gli episodi entrando in collisione con il fantasma.

Al contrario, Q-Learning e SARSA mantengono collision rate molto bassi, confermando la capacità di apprendere comportamenti di sopravvivenza efficaci.

7.7 Confronto tra Q-Learning e SARSA

Il confronto tra i due algoritmi evidenzia differenze coerenti con la teoria del Reinforcement Learning.

Nel training eseguito, SARSA ottiene prestazioni leggermente superiori in termini di reward media, numero medio di cibi raccolti e collision rate. Tuttavia, la differenza tra i due algoritmi è contenuta, indicando che entrambi riescono ad apprendere politiche efficaci per l'ambiente proposto. Questo comportamento è coerente con la natura off-policy dell'algoritmo, che tende a privilegiare strategie più aggressive e orientate alla massimizzazione della reward futura.

SARSA, invece, mostra un comportamento leggermente più prudente. Poiché aggiorna i valori Q considerando le azioni realmente selezionate dalla politica corrente, comprese quelle esplorative, tende a sviluppare strategie più conservative.

Nonostante queste differenze, entrambi gli algoritmi riescono ad apprendere politiche efficaci e a superare nettamente l'agente casuale.

Capitolo 8

Discussione dei Risultati

L'analisi dei risultati ottenuti mostra che gli algoritmi di Reinforcement Learning implementati riescono ad apprendere strategie efficaci all'interno dell'ambiente Pac-Man sviluppato.

Fin dalle prime fasi del training si osserva un miglioramento progressivo della reward media, accompagnato da un aumento del numero di cibi raccolti e da una riduzione delle collisioni con il fantasma. Questo comportamento conferma che gli agenti riescono progressivamente a comprendere la struttura dell'ambiente e a selezionare azioni sempre più vantaggiose.

8.1 Comportamento dell'agente casuale

L'agente casuale presenta prestazioni molto basse in tutte le metriche considerate. Il collision rate pari al 100% indica che l'agente termina praticamente ogni episodio entrando in collisione con il fantasma.

La reward media negativa e il numero ridotto di step sopravvissuti mostrano chiaramente che un comportamento puramente casuale non è sufficiente per affrontare efficacemente il problema.

Questo risultato rappresenta un utile baseline per valutare l'effettiva capacità di apprendimento degli algoritmi RL.

8.2 Apprendimento degli agenti RL

Sia Q-Learning sia SARSA riescono a sviluppare strategie efficaci. Gli agenti imparano progressivamente a:

- evitare il fantasma;
- minimizzare i movimenti inutili;
- raggiungere rapidamente il cibo;
- sopravvivere per un numero elevato di step.

L'elevata reward media ottenuta e il basso collision rate confermano la qualità delle politiche apprese.

I grafici di training mostrano inoltre che, nonostante la presenza di rumore nella reward dei singoli episodi, la media mobile presenta un andamento crescente abbastanza stabile. Questo comportamento è tipico degli ambienti stocastici nel Reinforcement Learning.

8.3 Interpretazione del Q-value

Durante il training è stato monitorato anche l'andamento del valore Q associato allo stato iniziale dell'episodio. Tale valore mostra una crescita progressiva nel corso degli episodi, indicando che la Q-table attribuisce stime sempre più elevate della reward futura attesa.

Tuttavia, il Q-value non converge perfettamente a un valore costante. Questo comportamento è plausibile e coerente con la struttura dell'ambiente utilizzato.

Infatti:

- la posizione del cibo varia casualmente a ogni episodio;
- il fantasma utilizza una politica di movimento parzialmente casuale;
- la politica ϵ -greedy mantiene una componente di esplorazione durante tutto il training.

Di conseguenza, lo stato iniziale non è identico in ogni episodio e le stime della reward futura rimangono soggette a variazioni.

Per questo motivo, il Q-value deve essere interpretato principalmente come un indicatore qualitativo dell'apprendimento della Q-table, piuttosto che come una misura esatta della convergenza dell'algoritmo.

8.4 Differenze tra Q-Learning e SARSA

Le differenze osservate tra Q-Learning e SARSA risultano coerenti con la teoria.

Q-Learning ottiene prestazioni leggermente superiori in termini di reward media e collision rate. Questo comportamento deriva dalla natura off-policy dell'algoritmo, che aggiorna i valori Q assumendo sempre l'azione ottimale nello stato successivo.

SARSA, essendo on-policy, aggiorna invece i valori Q considerando le azioni realmente selezionate dalla politica corrente. Di conseguenza, tende a sviluppare strategie più conservative e leggermente meno aggressive.

Nel contesto del progetto, entrambe le strategie si sono comunque dimostrate efficaci.

8.5 Limiti del progetto

Nonostante i risultati positivi, il progetto presenta alcune limitazioni.

L'ambiente utilizzato è relativamente piccolo e completamente osservabile. Inoltre, la Q-table cresce rapidamente all'aumentare della dimensione dello spazio degli stati, rendendo gli approcci tabulari poco scalabili per ambienti più complessi.

Un ulteriore limite riguarda l'utilizzo di un valore di ϵ costante durante tutto il training. L'introduzione di una strategia di epsilon decay potrebbe migliorare ulteriormente la convergenza degli algoritmi.

Infine, i risultati sono stati ottenuti eseguendo un singolo training per algoritmo. In contesti sperimentali più avanzati sarebbe opportuno effettuare più esecuzioni con seed differenti e analizzare media e deviazione standard delle metriche ottenute.

Capitolo 9

Conclusioni

In questo progetto è stato sviluppato un ambiente Pac-Man semplificato per studiare l'applicazione di tecniche di Reinforcement Learning basate su Q-table.

Sono stati implementati manualmente gli algoritmi Q-Learning e SARSA, senza utilizzare librerie esterne di Reinforcement Learning. Entrambi gli agenti sono stati addestrati attraverso episodi ripetuti, imparando progressivamente strategie efficaci per raccogliere il cibo ed evitare il fantasma.

I risultati sperimentali mostrano che entrambi gli algoritmi ottengono prestazioni significativamente superiori rispetto a un agente casuale. In particolare, gli agenti addestrati riescono a:

- aumentare la reward media;
- raccogliere una quantità maggiore di cibo;
- sopravvivere per un numero maggiore di step;
- ridurre drasticamente il numero di collisioni.

Nel training eseguito, SARSA ha ottenuto prestazioni leggermente superiori rispetto a Q-Learning, mostrando un comportamento più stabile e conservativo. Questo risultato è coerente con la natura on-policy dell'algoritmo SARSA, particolarmente adatto ad ambienti parzialmente stocastici.

L'analisi del Q-value iniziale ha mostrato un andamento crescente durante il training, indicando che gli agenti apprendono progressivamente una politica migliore. Tuttavia, il valore non converge perfettamente a una costante, a causa della presenza di esplorazione epsilon-greedy e della componente stocastica dell'ambiente.

Come possibile sviluppo futuro, il progetto potrebbe essere esteso utilizzando tecniche di Deep Reinforcement Learning, come Deep Q-Networks (DQN), per affrontare ambienti più complessi e spazi degli stati di dimensione maggiore.

Bibliografia

- [1] Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, Second Edition, in progress, 2014-2015.
- [2] Pygame Documentation, <https://www.pygame.org/docs/>
- [3] NumPy Documentation, <https://numpy.org/doc/>
- [4] Matplotlib Documentation, <https://matplotlib.org/stable/contents.html>
- [5] Francesco Pupo, *Lecture Notes of Intelligent Systems for Robotics*, University of Calabria.